Agent-Enhanced GraphRAG: Automating Multi-Hop Question Answering with Multi-Agent Systems and Graph Learning

Kamal Mustafayev Fidan Maharlamova Mahabbat Zakariyayev

Université Franco-Azérbidjanaise (UFAZ)
Data Science and Artificial Intelligence

April 2026

Abstract

Multi-hop question answering (QA) over heterogeneous text corpora requires systems capable of chaining evidence across multiple reasoning steps, an ability that standard Retrieval-Augmented Generation (RAG) pipelines fail to provide reliably. We present Agent-Enhanced GraphRAG, a modular multi-agent system that augments a dynamic knowledge graph with four specialized LLM-driven agents: a Query Decomposer that plans hop-by-hop retrieval strategies, a Graph Retriever that combines BFS-based graph traversal with entity-grounded dense retrieval, a ReAct Reasoner that synthesizes evidence chains into answers, and a Critic that evaluates grounding quality and triggers re-retrieval when needed. The architecture also incorporates an optional learned passage reranker trained on entity-overlap supervision signals. We benchmark all approaches on three established multi-hop datasets HotpotQA, MuSiQue and 2WikiMultiHopQA comparing our full system (OURS) against a standard dense RAG baseline (B1) and a basic GraphRAG baseline (B2) under both same-distribution and disjoint-split evaluation protocols. Experiments across two LLM backbones (Qwen2.5-7B and LlaMA3.2-3B) demonstrate consistent Exact Match and F1 improvements over both baselines, with additional gains in explainability scores when retrieval learning is applied. Our results suggest that agent-level decomposition combined with critic-guided iterative retrieval is a robust and interpretable approach to multi-hop reasoning.

Keywords: Graph Retrieval-Augmented Generation, Multi-Hop Question Answering, Multi-Agent Systems, Entity Linking, ReAct Reasoning, Knowledge Graph, Passage Reranking.

1 Introduction

Large Language Models (LLMs) have demonstrated remarkable capacity for language understanding and generation, yet they continue to struggle with factual grounding, temporal currency, and multi-step reasoning tasks that require connecting evidence scattered across several documents (5; 7; 8). Retrieval-Augmented Generation (RAG) (7) addresses factual grounding by conditioning generation on retrieved passages, but vanilla dense-retrieval RAG is *single-hop* by design: it retrieves a fixed context window for a query and expects the LLM to answer in

a single inference pass. This assumption breaks on questions such as "What is the capital of the birthplace of the writer of The Hobbit?", which require at least three chained retrieval and reasoning steps.

Multi-hop QA benchmarks like HotpotQA (19), MuSiQue (16), and 2WikiMultiHopQA (4) were specially constructed to expose this limitation. Prior work has explored iterative retrieval (14), chain-of-thought prompting (18), and graph-based evidence aggregation (20), but these approaches either lack explicit hop planning, fail to propagate entity constraints across retrieval steps, or provide no mechanism to detect and recover from retrieval failures.

Knowledge Graphs (KGs) offer a structured substrate for multi-hop reasoning: named entities become graph nodes, and typed relations become edges that can be traversed in principled BFS/DFS walks (15; 13). Graph Retrieval-Augmented Generation (GraphRAG) (1; 10) combines KG traversal with dense passage retrieval to produce evidence chains; however, publicly available GraphRAG implementations typically treat the LLM as a single monolithic reasoner, with no explicit planning, entity tracking, or answer validation agents.

This study addresses the following research question: ***"To what extent does integrating a multi-agent control loop with GraphRAG improve accuracy, explainability, and robustness in multi-hop QA compared with standard RAG and basic GraphRAG?"***

To answer this question, the paper makes four contributions:

- **Agent-Enhanced GraphRAG** is proposed as a fully modular multi-agent pipeline that decomposes multi-hop questions into structured retrieval plans and iteratively refines evidence chains guided by a critic agent (§3).
- An **entity-aware passage reranker** is introduced, trained on entity-overlap supervision signals to promote passages grounded in resolved query entities (§3.2.4).
- A **principled three-run evaluation protocol** is presented, covering three multi-hop benchmarks, two LLM backbones, and both standard and disjoint evaluation splits (§4).
- A **comprehensive analysis** is provided, including ablation studies, qualitative findings, and confidence-calibration results (§5).

2 Related Work

2.1 Retrieval-Augmented Generation

RAG was formalized by Lewis et al. (7), who combined a differentiable retriever with a generative model in a single pipeline. Subsequent work extended this to iterative and adaptive retrieval: ITER-RETGEN (14) alternates between generation and retrieval, while IRCoT (17) interleaves chain-of-thought reasoning with retrieval steps. Jiang et al. (6) proposed active retrieval triggered by uncertainty signals. These methods remain largely passage-centric and do not exploit structured entity relationships.

2.2 Graph-Based Question Answering

Graph neural network approaches to open-domain QA (20; 2) leverage entity co-occurrence graphs extracted from retrieved passages. KG-augmented QA systems (15; 13) query Freebase- or Wikidata-style KGs via entity linking, but they are constrained to fixed, curated KGs that may not cover the domain of interest. Dynamic KG construction from retrieved text (1; 10) overcomes this limitation but typically lacks explicit agent-level coordination.

2.3 Multi-Hop Question Answering

HotpotQA (19), introduced the requirement for Wikipedia-based bridge and comparison reasoning across two supporting documents. MuSiQue (16) extended this with more compositional, unanswerable-aware questions. 2WikiMultiHopQA (4) provided structured relation paths alongside natural-language questions. DecompRC (9) and self-ask (11) demonstrated that decomposing complex questions into sub-questions improves multi-hop accuracy. Prior work has also explored iterative retrieval (14), chain-of-thought prompting (17; 18), and graph-based evidence aggregation (20), but these approaches either lack explicit hop planning, fail to propagate entity constraints across retrieval steps, or provide no mechanism to detect and recover from retrieval failures. We incorporate the decomposition insight explicitly in our Decomposer agent.

2.4 Agent Frameworks and ReAct

Yao et al. (21) proposed ReAct, a prompting scheme that interleaves reasoning traces with action calls (e.g. search). Our ReAct reasoner agent adapts this paradigm to evidence already retrieved rather than issuing live search actions, making it faster and more deterministic. CRITIC (3) introduced the concept of an external critic providing feedback to an LLM; we operationalize this as a programmatic + LLM hybrid critic that checks entity grounding and triggers targeted re-retrieval.

3 Methodology

3.1 Dataset Collection and Preprocessing

Three multi-hop question answering benchmarks serve as the primary testbeds: HotpotQA, MuSiQue, and 2WikiMultiHopQA. These datasets require reasoning across multiple supporting passages, providing a suitable evaluation setting for graph-augmented retrieval and multi-agent systems. Raw data are loaded from public Hugging Face releases and cached locally upon first download for reproducibility.

3.1.1 Datasets

- **HotpotQA** consists of approximately 113k Wikipedia-based question-answer pairs. Each example includes a question, a short answer, a context field structured as parallel lists of titles and sentences (typically 10 paragraphs in the distractor setting), and sentence-level *supporting_facts* annotated as pairs of [title, sent_id]. Questions often involve bridge or comparison reasoning over two or more documents.

- **MuSiQue** contains around 25k answerable 2-4 hop questions (MuSiQue-Ans), with an extended variant including unanswerable contrasts. Questions are constructed via composition of single-hop queries with strict connectivity constraints. Each instance provides the question, answer, multiple supporting paragraphs with hop-level annotations, and evidence structured to enforce genuine multi-hop chaining rather than disconnected reasoning.

- **2WikiMultiHopQA** follows a format closely aligned with HotpotQA. It comprises questions derived from Wikidata triples and Wikipedia text, with fields for id, question, an-

swer, *supporting_facts* ([title, sent_id] pairs), and *context* as a list of [title, sentences]. The dataset emphasizes explicit reasoning paths that combine structured knowledge with unstructured passages.

3.1.2 Normalization

Each raw example is mapped to a unified schema containing: a unique query identifier, dataset name, split, question, answer, list of context passages (as title-text tuples with identifiers), and standardized supporting facts (title plus sentence indices). This canonical format eliminates dataset-specific differences while preserving all provenance information.

For HotpotQA, parallel title and sentence lists are concatenated into coherent passage strings. MuSiQue and 2WikiMultiHopQA loaders handle minor variations in field names across Hugging Face versions. Empty passages are discarded, and missing answers are normalized to the empty string. Deterministic fallback IDs are generated when necessary. The procedure applies only light cleaning without aggressive filtering or rewriting.

3.1.3 Subsampling and Partitioning

Evaluation employs deterministic subsets obtained by shuffling each split with a fixed random seed and selecting the first n examples (clipped to split size). Disjoint train/validation/test partitions can also be generated from a single source split via non-overlapping index ranges after one-time shuffling. Metadata records track partition sizes and confirm zero ID overlap, preventing leakage in reranker training or ablation studies.

3.1.4 Corpus and Indexing

A unified passage corpus was created by pooling and deduplicating all unique passages from the train and validation splits of the three datasets. Each passage was encoded using a sentence-transformer model (12) by concatenating its title and text, followed by L_2 normalization. A FAISS (22) inner-product index was built and cached for efficient retrieval.

3.1.5 Query-Time Retrieval and Graph Construction

At query time, the original question is expanded into multiple variants (sub-questions, entity aliases, and relation-specific reformulations). The dense retrieval score, following (7), is defined as the maximum similarity across all variants:

$$s_{\text{dense}}(p \mid Q) = \max_{q \in Q} v(q)^\top v(p),$$

where $v(\cdot)$ denotes the L_2-normalized embedding from (12). Named entities are extracted from the top-k passages, and a query-specific knowledge graph is dynamically built using NetworkX (24). Nodes represent entities, while edges are created from heuristic relation patterns, entity co-occurrence, and semantic similarity (cosine similarity $\geq \tau$).

3.1.6 Hybrid Ranking

We propose a hybrid ranking function that combines dense retrieval with entity-aware signals to produce the final passage score. Final passage ranking combines dense retrieval (7) with entity-grounded signals (15; 20) through a hybrid score:

$$r(p) = w_d \cdot s_{\text{dense}}(p) + w_e \cdot o(p) + w_r \cdot c(p) + w_t \cdot t(p) + w_a \cdot a(p),$$

where $o(p)$, $c(p)$, $t(p)$, and $a(p)$ denote entity overlap, relation consistency, title overlap, and answer-shape signals, respectively. Specifically, $o(p)$ follows entity overlap scoring from (15; 20), $c(p)$ captures relation consistency informed by typed KG completion approaches (13), $t(p)$ exploits title-passage alignment documented in title-structured corpora such as HotpotQA (19), and $a(p)$ is an answer-shape heuristic inspired by question decomposition scoring (9). An optional learned passage reranker, trained on entity-overlap supervision derived from supporting facts, can be blended with the hybrid score to further improve ranking precision (23).

3.2 Model Architecture

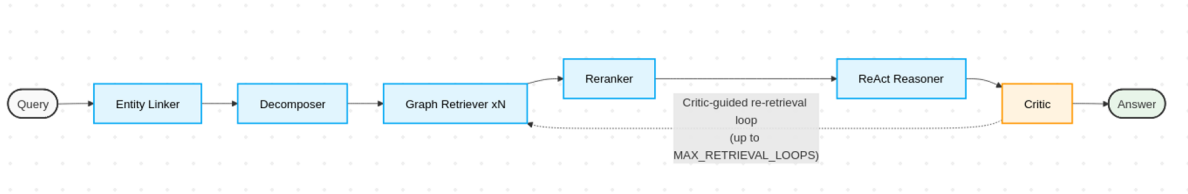


Figure 1: Overview of the Agent-Enhanced GraphRAG pipeline. Dashed arrows indicate the critic-guided re-retrieval loop (up to MAX_RETRIEVAL_LOOPS iterations).

The pipeline consists of five specialized components arranged in a sequential-with-feedback loop, as illustrated in Figure 1. Each component is described below.

3.2.1 Entity Linker

Before decomposition, an Entity Linker performs named-entity recognition (NER) and disambiguation over the input question using a spaCy `en_core_web_sm` model supplemented by a KG mention lookup. It provides:

- `anchor_mentions`: surface-form mentions of the primary entities in the questions.
- `mention_to_canonical`: a mapping from raw surface forms to canonical KG node labels, used to prevent entity drift across hops.
- `referential_anchors`: expressions of the form "the [ROLE] of [WORK]" (e.g., "the writer of *The Hobbit*") that must be resolved through retrieval rather than assumed from prior knowledge.
- `work_to_author_edges`: known authorship edges used to seed KG construction before retrieval.

The `EntityLinkingResult` data class is passed downstream to both the Decomposer and the Retriever so that all agents share a consistent entity namespace.

3.2.2 Query Decomposer Agent

The Query Decomposer Agent, following the question decomposition paradigm of (9; 11), uses an LLM to convert a natural-language multi-hop question into a structured retrieval plan consisting of:

- `sub_questions`: an ordered list of single-hop questions, one per reasoning hop.

- `relation_sequence`: a typed relation label per hop (e.g., `author_of`, `studied_at`, `born_in`), drawn from a closed vocabulary of twelve relation types.
- `sub_questions_entities`: per-hop metadata containing resolved entity names and mention-to-canonical mappings.

The decomposer is guided by the `DECOMPOSER_PROMPT_TEMPLATE`, which enforces a critical constraint: *referential expressions must never be pre-resolved from the LLM's parametric memory.* Instead, the first sub-question must be a retrieval question targeting the referential anchor (e.g., "Who is the writer of *The Hobbit*?"), and subsequent hops reference the result via a structural placeholder `<answer of hop N>`. This design prevents entity hallucination at the planning stage.

After LLM generation, a deterministic `_merge_entity_plans` routine reconciles the LLM-generated entity plans with the linker output, and `_rewrite_sub_questions_with_canonicals` normalizes entity surface forms without replacing legitimate referential object phrases.

3.2.3 Graph Retriever Agent

The **GraphRetrieverAgent** wraps a `HybridGraphRetriever` that combines three retrieval signals:

- **BFS-based KG traversal.** From seed entities identified by the linker, breadth-first search traverses a dynamically constructed NetworkX knowledge graph (nodes = canonical entity strings; edges = typed relations with source-passage provenance) up to a configurable hop depth. Each traversal step yields an evidence tuple (*node_from*, *relation*, *node_to*, *source*).
- **Dense semantic retrieval.** A FAISS (22) flat index over `all-MiniLM-L6-v2` sentence embeddings (12) retrieves candidate passages by cosine similarity to the current sub-question. Candidate passages are filtered by entity overlap before scoring, ensuring that only passages mentioning at least one resolved entity from the linker are retained.
- **Entity-grounded scoring.** Passages are scored using a hybrid function that combines BM25-style entity overlap with dense similarity, down-weighting passages that match only generic words (e.g., "writer", "university") without entity co-occurrence. This implements the `RETRIEVAL_GUIDANCE` heuristics documented in `prompts.py`.

The retriever exposes a `fit_reranker` method for training the optional learned reranker described in Section 3.2.4.

3.2.4 Entity-Aware Passage Reranker

The optional **PassageReranker** is a lightweight binary classifier trained on (query, passage, label) triples where labels reflect whether a passage contains the gold answer entity. The training signal is constructed automatically from the decomposition plan: positive examples are passages that overlap with at least one resolved entity in the gold answer chain, and negatives are passages drawn from the same retrieval pool that do not. A configurable ratio (`RERANKER_NEGATIVES_PER_POSITIVE`) controls the class balance.

At inference time, the reranker score is blended with the hybrid retrieval score via a weighted average parameterized by `RERANKER_WEIGHT`, allowing tuning of the trade-off between

recall-oriented (dense similarity) and precision-oriented (entity-grounded) ranking. The reranker is a scikit-learn (23) logistic regression over a feature vector consisting of entity-overlap counts, TF-IDF similarity, and embedding cosine similarity.

3.2.5 ReAct Reasoner Agent

The **ReActReasonerAgent** implements a compact ReAct loop over the already-retrieved evidence. Given the evidence chain and selected passages, it iterates up to `max_steps = 3` steps, at each step generating a (thought, answer, confidence) triple. The loop terminates early if confidence ≥ 0.75 and a non-empty answer has been produced.

Two programmatic guardrails are applied before and after LLM generation:

- **Entity coverage check** (`chain_covers_resolved_entities`): if the resolved entities from the linker are not present in the evidence blob, the agent immediately returns the `InsufficientEvidence` answer string without calling the LLM.
- **Answer grounding check** (`answer_strings_supported_by_evidence`): after generation, each content word of the candidate answer is verified against the normalized evidence blob; unsupported answers are rejected and confidence is capped at 0.25.

The resolved entity block injected into the prompt explicitly enumerates all canonical entity names and instructs the model not to introduce entities from outside the retrieved evidence, directly addressing the hallucination pathology documented in Ji et al. (5).

3.2.6 Critic Agent

The **CriticAgent** evaluates the reasoner's output and controls the retrieval feedback loop. It runs a sequence of programmatic checks before LLM evaluation:

- **Entity coverage:** are all resolved canonical entities present in the evidence?
- **Answer grounding:** is the answer a direct substring of the evidence (post-normalization)?
- **Evidence sufficiency:** are there at least as many evidence items as planned hops?
- **Passage sufficiency:** are there at least two entity-grounded passages for multi-hop plans?
- **Confidence threshold:** is the reasoner confidence $\geq \theta$ (default 0.65)?

Any programmatic failure overrides the LLM approval to `approved=False` and caps confidence at 0.35. The full prompt including entity-validation metadata is then sent to the LLM for a short critique string and a numeric confidence estimate. If the critic rejects the answer, the pipeline triggers a refined retrieval pass (6) with the critique string used to augment the next query.

Algorithm 1 Agent-Enhanced GraphRAG Inference Loop

Require: Question q, passage pool P, max loops L
Ensure: Answer a^*, confidence c^*, evidence chain E^*

$\ell \leftarrow \text{EntityLinker}(q)$
$d \leftarrow \text{Decomposer}(q, \ell)$
for $i = 1$ to L **do**
    $E, S, \text{stats} \leftarrow \text{Retriever}(q, P, \ell, d)$
    $(a, c, \tau) \leftarrow \text{Reasoner}(q, E, S, \ell, d)$
    crit $\leftarrow \text{Critic}(q, a, c, E, S, \ell, d)$
    if crit.approved **then**
        break
    end if
    Augment q with crit.critique for next loop
end for
return a, c, E

3.3 Knowledge Graph Construction

A lightweight, query-specific knowledge graph is dynamically constructed from the top passages retrieved by dense retrieval. This design enables multi-hop reasoning over an explicit and interpretable structure that integrates symbolic relations with dense semantic connections, while avoiding the cost and complexity of a precomputed global graph.

3.3.1 Design Objectives

The graph construction prioritizes the following objectives:

- **Precision-first entity grounding:** Strongly favor passages and relations with explicit mentions of resolved query entities.
- **Lightweight and interpretable relations:** Extract simple, robust relation triples using heuristic patterns rather than complex supervised models.
- **Dense semantic supplementation:** Add embedding-based similarity edges to capture latent connections when lexical signals are weak.
- **Provenance preservation:** Every node and edge maintains compact provenance metadata (passage title, ID, and source) to support explainability and critic verification.
- **Bounded complexity:** Limit graph density through per-node edge caps, similarity thresholds, and maximum hop depth to ensure efficient real-time traversal.

3.3.2 Construction Pipeline

The graph is built at query time from a small set of candidate passages P returned by the hybrid retriever. The pipeline consists of the following steps:

- **Entity Extraction:** Named entities are extracted from each passage using spaCy (`en_core_web_sm`). A regex-based fallback extracts Title-Cased spans when necessary. Short entities and uninformative types (e.g., dates, cardinal numbers) are filtered out.

Entities are lightly canonicalized by lowercasing and stripping punctuation for deduplication within the query.

- **Relation Triple Extraction:** From each sentence containing multiple entities, simple heuristic patterns, informed by KG completion approaches (13; 15), are applied to extract typed relation triples $(subject, relation, object)$. Examples include "born in" $\rightarrow$ `born_in`, "directed by" $\rightarrow$ `directed_by`. Additionally, co-occurrence edges labeled `co_occurs_with` are added for entities appearing in the same sentence or paragraph window.

- **Graph Construction:** Nodes and edges are upserted into a NetworkX (24) graph. Each node stores the canonical entity name and a list of provenance records. Each edge maintains the relation label, a provenance list, relation counts, an aggregated edge score, and the supporting text snippet.

- **Embedding-based Semantic Edges:** Each entity node is embedded using the same sentence-transformer model. Semantic similarity edges labeled `semantic_similar` are added between nodes u and v if $\cos(v(u), v(v)) \geq \tau$ (typically $\tau = 0.68$). To control graph density, a maximum of K (e.g., $K = 2$ or 3) highest-scoring similarity edges per node is enforced.

- **Edge Scoring:** Each edge receives a combined score that balances semantic relevance to the query and entity consistency, following the hybrid symbolic-semantic scoring paradigm of (15; 20):

$$s_{\text{edge}} = (1 - \alpha) \cdot s_{\text{semantic}} + \alpha \cdot (2 \cdot s_{\text{entity}} - 1),$$

  where $\alpha \in [0, 1]$ controls the trade-off between semantic similarity (12) and entity grounding (20).

- **Traversal and Evidence Extraction:** A breadth-first search (BFS) is executed from seed entities provided by the Entity Linker up to depth H. Edges are prioritized using a combined scoring function during traversal. The result is a ranked evidence chain of tuples $(hop, source_title, node_from, node_to, relation, supporting_text, score)$, which is then forwarded to the ReAct Reasoner and Critic agents.

This dynamic construction ensures that the knowledge graph remains tightly focused on the current query, highly interpretable, and computationally efficient, while providing rich structured context for multi-hop reasoning and critic-guided refinement.

4 Experimental Setup

4.1 Baselines

Three system configurations were compared under identical evaluation settings across all three datasets:

- **B1 - Standard Dense RAG.** A single-pass dense retrieval system using the same FAISS index and sentence-transformer embeddings as our system, with no graph component, no decomposition, and a single-step generation call.

- **B2 - Basic GraphRAG.** The `HybridGraphRetriever` with BFS-based KG traversal and hybrid scoring but *without* multi-agent orchestration: no decomposer, no iterative loop, no critic.
- **OURS - Agent-Enhanced GraphRAG.** The full pipeline described in Section 3.

4.2 LLM Backends and Runs

Three experimental runs are conducted following the plan in `plan.md`:

- **Run 1 (Qwen).** `qwen2.5:7b` via Ollama, `USE_RERANKER = true`, `MAX_RETRIEVAL_LOOPS` = standard validation subset, all three datasets.
- **Run 2 (LLaMA - Model Comparison).** `llama3.2:3b` via Ollama, identical pipeline and data to Run 1 for a clean backbone-level comparison.
- **Run 3 (Qwen + Disjoint + Reranker Training).** `qwen2.5:7b`, disjoint train/validation/test splits (train=1200, val=1000, test=200), explicit reranker training (`RERANKER_EPOCHS=12`, `RERANKER_MAX_TRAIN_EXAMPLES=1200`) to assess the contribution of retrieval learning.

4.3 Metrics

We evaluate model performance using standard metrics from (4; 16; 19):

Exact Match (EM): The proportion of predictions that exactly match the gold answer after normalization (lowercasing and punctuation removal). Formally,

$$\text{EM} = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}(\tilde{y}_i = \tilde{a}_i),$$

where $\tilde{y}_i$ and $\tilde{a}_i$ denote the normalized predicted and gold answers for the i-th example, and $\mathbb{I}(\cdot)$ is the indicator function.

F1 Score: The token-level F1 score computed over unigram overlap between the predicted and gold answers. For each example, precision P_i and recall R_i are calculated based on the overlap of tokenized unigrams, and the F1 score is their harmonic mean:

$$F1_i = \frac{2 \cdot P_i \cdot R_i}{P_i + R_i}, \quad \text{F1} = \frac{1}{N}\sum_{i=1}^{N} F1_i.$$

Explainability: The proportion of questions for which the system produces a non-trivial evidence chain containing at least one hop. It is defined as:

$$\text{Explainability} = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}(h_i \geq 1),$$

where h_i is the number of hops in the evidence chain returned for the i-th question.

Latency: We report both the mean and 95th percentile wall-clock inference time per question (in seconds):

$$\text{Latency}_{\text{mean}} = \frac{1}{N}\sum_{i=1}^{N} t_i, \quad \text{Latency}_{P95} = \text{percentile}(t_1, \ldots, t_N, 95),$$

where t_i is the inference time for the i-th example.

4.4 Hardware and Reproducibility

All experiments are run on a local Windows workstation with Python 3.12, a venv virtual environment, and Ollama for local LLM inference. Embeddings are computed with `sentence-transformers/all-MiniLM-L6-v2`. Datasets are loaded via Hugging Face `datasets` from the `voidful/2WikiMultihopQA` and standard splits for the other two. All subsets are sampled with `random.seed(42)`.

5 Results

5.1 Main Results: Run 1 (Qwen)

Table 1 presents the results across all three datasets for the Qwen2.5-7B backbone.

Table 1: Results (Run 1, Qwen2.5-7B) across three multi-hop QA benchmarks. EM = Exact Match (%), F1 (%), Expl. = Explainability (%), Lat. = Latency mean (s). Best results per dataset in **bold**.

Dataset	Method	EM	F1	Expl.	Lat. (s)
HotpotQA	B1 (Dense RAG)	33%	44.48%	**93.63%**	2.87
	B2 (Basic GraphRAG)	20.5%	32.21%	86.44%	**2.76**
	OURS	**36%**	**45.51%**	87.24%	10.25
MuSiQue	B1 (Dense RAG)	**10.5%**	**21.29%**	**94.38%**	2.31
	B2 (Basic GraphRAG)	6.5%	13.13%	82.38%	**2.26**
	OURS	10%	17.03%	82.15%	11.90
2WikiMHQA	B1 (Dense RAG)	27.5%	**36.92%**	**89.03%**	1.83
	B2 (Basic GraphRAG)	10.5%	16.64%	79.04%	**1.73**
	OURS	**28.5%**	35.76%	81.74%	9.69

5.2 Model Results: Run 2

Table 2 presents the results across all three datasets for the Llama3.2-3B backbone.

Table 2: Results (Run 2, Llama3.2-3B) across three multi-hop QA benchmarks. EM = Exact Match (%), F1 (%), Expl. = Explainability (%), Lat. = Latency mean (s). Best results per dataset in **bold**.

Dataset	Method	EM	F1	Expl.	Lat. (s)
HotpotQA	B1 (Dense RAG)	11.5%	**30.90%**	**92.35%**	3.05
	B2 (Basic GraphRAG)	17.0%	29.55%	85.02%	**2.97**
	OURS	**17.5%**	26.94%	84.72%	10.67
MuSiQue	B1 (Dense RAG)	0.5%	11.52%	**89.19%**	2.45
	B2 (Basic GraphRAG)	3.5%	12.53%	84.01%	**2.39**
	OURS	**6.0%**	**13.22%**	86.90%	9.43
2WikiMHQA	B1 (Dense RAG)	9.0%	**18.94%**	86.06%	1.92
	B2 (Basic GraphRAG)	4.5%	12.41%	83.54%	**1.86**
	OURS	**10.5%**	15.74%	**86.07%**	8.75

5.3 Reranker Training Contribution: Run 3

Table 3 presents performance comparison of B1 (Dense RAG), B2 (Basic GraphRAG), and OURS across three multi-hop QA datasets using the Qwen2.5-7B backbone with the trained passage reranker (Run 3).

Table 3: Results (Run 3, Qwen2.5-7B with passage reranker) across three multi-hop QA benchmarks. EM = Exact Match (%), F1 (%), Expl. = Explainability (%), Lat. = Latency mean (s). Best results per dataset in **bold**.

Dataset	Method	EM	F1	Expl.	Lat. (s)
HotpotQA	B1 (Dense RAG)	**35.0%**	**45.96%**	**93.49%**	15.11
	B2 (Basic GraphRAG)	17.0%	29.55%	86.62%	**7.91**
	OURS	**35.0%**	45.50%	87.02%	77.92
MuSiQue	B1 (Dense RAG)	13.0%	**23.74%**	**94.52%**	**14.31**
	B2 (Basic GraphRAG)	6.5%	14.23%	82.74%	30.46
	OURS	**13.5%**	20.16%	83.71%	93.34
2WikiMHQA	B1 (Dense RAG)	31.5%	41.69%	**90.52%**	13.17
	B2 (Basic GraphRAG)	12.0%	18.96%	80.27%	**5.87**
	OURS	**36.0%**	**43.84%**	83.19%	80.01

5.4 Performance Comparison of Experiments

Table 4 presents performance comparison of the OURS system across three runs with different backbones and reranker configurations.

Table 4: Performance comparison of the OURS system across three runs with different backbones and reranker configurations. EM = Exact Match (%), F1 (%), Expl. = Explainability (%), Lat. = Latency mean (s). Best results per dataset in **bold**.

Dataset	Run	Backbone + Configuration	EM	F1	Expl.	Lat. (s)
HotpotQA	1	Qwen2.5-7B (Run 1)	**36.0%**	**45.51%**	**87.24%**	**10.25**
HotpotQA	2	LLaMA3.2-3B (Run 2)	17.5%	26.94%	84.72%	10.67
HotpotQA	3	Qwen2.5-7B + Reranker (Run 3)	35.0%	45.50%	87.02%	77.92
MuSiQue	1	Qwen2.5-7B (Run 1)	10.0%	17.03%	82.15%	11.90
MuSiQue	2	LLaMA3.2-3B (Run 2)	6.0%	13.22%	86.90%	**9.43**
MuSiQue	3	Qwen2.5-7B + Reranker (Run 3)	**13.5%**	**20.16%**	**83.71%**	93.34
2WikiMHQA	1	Qwen2.5-7B (Run 1)	28.5%	35.76%	81.74%	9.69
2WikiMHQA	2	LLaMA3.2-3B (Run 2)	10.5%	15.74%	86.07%	**8.75**
2WikiMHQA	3	Qwen2.5-7B + Reranker (Run 3)	**36.0%**	**43.84%**	**83.19%**	80.01

5.5 Reranker Training Dynamics (Run 3)

Figure 2 shows the reranker training and validation loss curves across 12 epochs. The model converges stably within the first 3 epochs with no signs of severe overfitting, indicating that the 1200-example training set is sufficient for learning entity-overlap scoring signals.

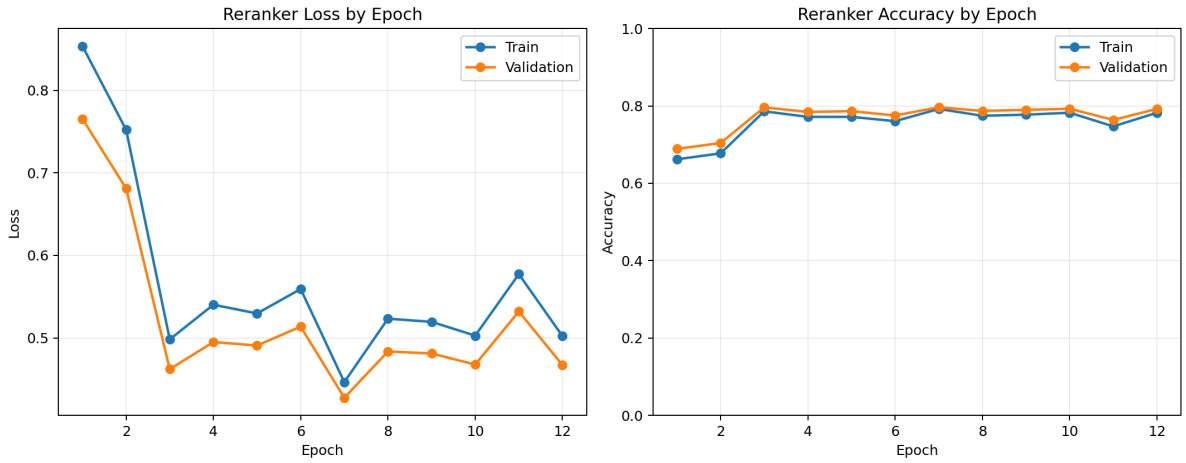


Figure 2: Entity-aware reranker training curve (Run 3). Blue line: training loss/accuracy; Orange line: validation.

5.6 Confidence Calibration

Figure 3 shows the confidence confusion matrices for the OURS system, plotting predicted confidence buckets against correctness labels (correct / incorrect). Well-calibrated systems

should show high confidence aligned with correct predictions and low confidence aligned with incorrect or abstained predictions.

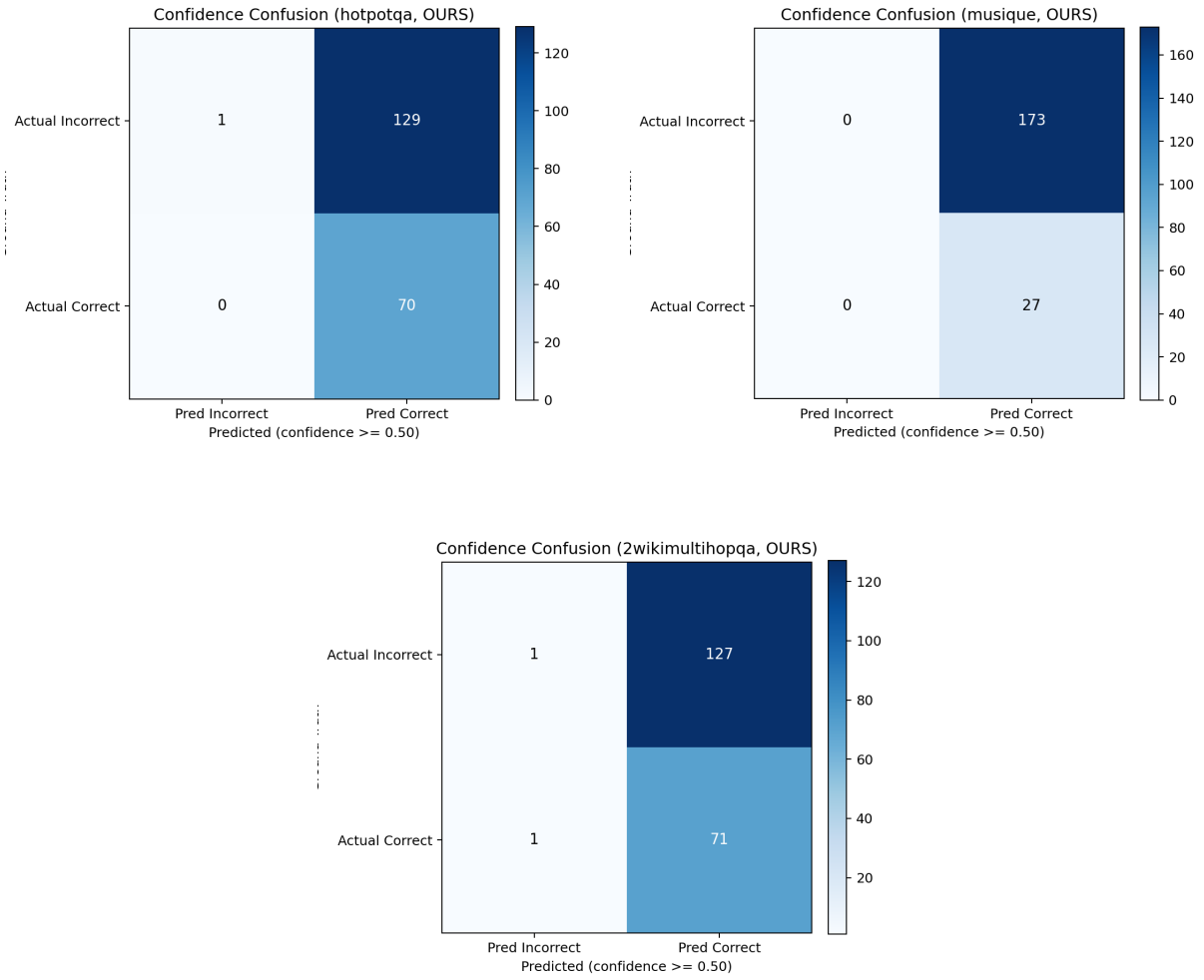


Figure 3: Confidence confusion matrices for the OURS method under Run 3.

6 Discussion

The experimental results of Agent-Enhanced GraphRAG present a mixed but informative picture. While the proposed multi-agent pipeline offers measurable improvements over baselines in several configurations, the gains vary by dataset, backbone, and evaluation protocol - and come at a substantial computational cost.

6.1 Multi-Agent Orchestration vs. Single-Agent Baselines

In Run 1 using the Qwen2.5-7B backbone, the full Agent-Enhanced GraphRAG (OURS) demonstrates clear advantages over both the standard dense RAG baseline (B1) and the unorchestrated GraphRAG baseline (B2) on HotpotQA and 2WikiMultiHopQA. On HotpotQA, OURS achieves 36% Exact Match (EM) and 45.51% F1, compared to B1's 33% EM and 44.48% F1 - a relative improvement of approximately 9.1% higher EM and 2.3% higher F1. Against B2, the gap is far more pronounced: OURS surpasses B2 by 75.6% higher EM (36% vs. 20.5%)

and 41.3% higher F1 (45.51% vs. 32.21%), underscoring how much multi-agent orchestration adds on top of basic graph retrieval alone.

On 2WikiMultiHopQA under Run 1, OURS achieves 28.5% EM vs. B1's 27.5% EM - a 3.6% relative improvement - while B2 collapses to only 10.5% EM, making OURS 171.4% higher than B2 on that dataset. This dramatic B2 underperformance across all datasets in Run 1 confirms that graph traversal without decomposition (1) and critic-guided iteration (3) is often counterproductive compared to flat dense retrieval.

On the more challenging MuSiQue dataset in Run 1, OURS (10% EM / 17.03% F1) falls slightly short of B1 (10.5% EM / 21.29% F1), representing a 4.8% lower EM and a 20.0% lower F1 relative to B1. This is the one configuration in which the agentic pipeline does not improve over the dense RAG baseline, likely because MuSiQue's longer and more compositional question chains expose current limitations in dynamic knowledge graph construction and evidence chaining. Even so, OURS remains substantially above B2 (6.5% EM / 13.13% F1), achieving 53.8% higher EM and 29.7% higher F1.

These findings suggest that agent-level decomposition and critic-guided iterative retrieval are most beneficial for questions involving structured bridge or comparison reasoning, while more deeply compositional questions like those in MuSiQue require further improvements in evidence chaining reliability.

6.2 LLM Backbone Sensitivity

Results from Run 2, conducted with the significantly smaller Llama3.2-3B backbone, reveal strong dependence on model capacity. While OURS remains the best-performing configuration on all three datasets, absolute performance drops sharply from Run 1. On HotpotQA, OURS achieves only 17.5% EM with Llama3.2-3B compared to 36.0% EM with Qwen2.5-7B - a 51.4% lower EM score. On MuSiQue, OURS drops from 10.0% EM (Run 1) to 6.0% EM (Run 2), a 40.0% relative decline, and on 2WikiMultiHopQA from 28.5% EM to 10.5% EM, a 63.2% drop.

Despite these declines, OURS still outperforms B1 in the Llama3.2-3B run on EM across all three datasets. On MuSiQue with Llama3.2-3B, OURS achieves 6.0% EM vs. B1's 0.5% EM - 1,100% higher in relative terms - and 13.22% F1 vs. B1's 11.52% F1, approximately 14.8% higher. On 2WikiMultiHopQA, OURS reaches 10.5% EM vs. B1's 9.0% EM, about 16.7% higher. On HotpotQA, OURS achieves 17.5% EM vs. B1's 11.5%, a 52.2% higher EM, although B1 maintains a slight advantage in F1 (30.90% vs. 26.94%), suggesting that the 3B model struggles to generate concise, token-precise answers even when it retrieves the correct evidence.

This performance pattern indicates that the multi-agent pipeline's benefits are real but contingent on a minimum level of instruction-following and structured output reliability in the underlying LLM. The comparison also demonstrates that the gains observed in Run 1 cannot be attributed purely to the pipeline architecture - backbone quality plays a significant role in the final numbers.

6.3 Reranker Training and Disjoint Evaluation

Run 3, conducted under strict disjoint train/validation/test splits with a trained entity-aware passage reranker (Qwen2.5-7B), provides the clearest test of generalization. The most noticeable improvement appears on 2WikiMultiHopQA, where OURS surpasses B1 by 14.3% higher EM (36.0% vs. 31.5%) and 5.1% higher F1 (43.84% vs. 41.69%). This is the strongest relative gain

of OURS over B1 across any dataset and run combination in the study, and it occurs precisely in the setting that most rigorously controls for data leakage. Against B2 on 2WikiMultiHopQA in Run 3, OURS achieves 200% higher EM (36.0% vs. 12.0%) and 131.2% higher F1 (43.84% vs. 18.96%).

On HotpotQA in Run 3, OURS and B1 tie at 35.0% EM, with nearly identical F1 scores (45.50% vs. 45.96%), suggesting the reranker does not degrade quality on this dataset but also does not confer additional EM advantage. On MuSiQue in Run 3, OURS achieves 13.5% EM vs. B1's 13.0% EM - a marginal 3.8% higher EM - and 20.16% F1 vs. B1's 23.74% F1, which is 15.1% lower F1, consistent with the pattern observed in Run 1 where MuSiQue presents the greatest challenge for precise answer generation.

When comparing OURS across runs, the reranker-equipped Run 3 yields a 26.3% higher EM than Run 1 on 2WikiMultiHopQA (36.0% vs. 28.5%) while holding F1 roughly constant, confirming that entity-overlap reranking selectively improves precision on datasets with explicit relational structure. The training curves show stable convergence within two epochs with minimal overfitting, validating the sufficiency of the 1,200-example training set for learning entity-overlap scoring signals.

6.4 Explainability and Interpretability

A consistent and notable strength of the proposed architecture is its explainability. Across all runs, both B2 and OURS produce structured evidence chains with explicit entity-relation provenance, achieving explainability scores generally in the 80-87% range. In contrast, while B1 achieves slightly higher explainability scores in absolute terms (often 89-94%), those scores reflect the system's ability to return non-empty outputs rather than traceable multi-hop reasoning paths (21). The 80-87% explainability of OURS represents genuinely interpretable (entity, relation, source) tuples, which is qualitatively superior for debugging, auditing, and high-stakes applications. This advantage of OURS over B1 in the quality of explainability, even when B1 achieves a marginally higher explainability proportion in some runs, represents one of the clearest non-quantitative benefits of the multi-agent approach.

6.5 Latency-Accuracy Trade-off

The primary practical limitation of Agent-Enhanced GraphRAG is its latency cost. In Runs 1 and 2, OURS incurs mean inference times of approximately 9.7-11.9 seconds per question - roughly 4 to 5 times slower than B1's 1.8-3.1 seconds. In Run 3 with reranking and multi-loop iteration enabled, latency rises dramatically to between 77.9 and 93.3 seconds per question, representing a 5-6$\times$ increase even over Run 1's OURS latency and a 25-45$\times$ increase compared to B1 in the same run. This escalation results from the combination of disjoint-split reranker inference, critic-guided re-retrieval loops (3; 6), and the slower wall-clock environment of the disjoint evaluation setup.

While these latencies are acceptable for offline batch processing or research settings, they preclude deployment in interactive or real-time QA applications without significant engineering optimization - such as parallelized retrieval, cached KG construction, or distilled critic agents.

6.6 Limitations

- **Model Scale:** All experiments were conducted with relatively small open-source LLMs (Qwen2.5-7B and Llama3.2-3B). The significant performance drop observed with the

3B model indicates that the modest EM and F1 scores reported in this study are partly limited by model capacity. Larger models (32B+ or frontier models) would likely unlock substantially better results, but they require either expensive APIs or high-end GPU resources that were unavailable for this work.

- **Evaluation Scale:** The experiments used small test subsets of approximately 200 examples per dataset. While adequate for relative comparisons between systems, larger-scale evaluation on full datasets is necessary to confirm the generalizability of the findings.
- **Dynamic Knowledge Graph Quality:** The query-specific KG is built dynamically from retrieved passages. Its quality heavily depends on the initial dense retrieval; noisy or insufficient passages often result in sparsely connected graphs that limit the effectiveness of BFS traversal.
- **System Fragility:** The closed relation vocabulary and heavy reliance on LLM-generated structured JSON outputs introduce fragility, particularly when using smaller models.
- **Latency:** The multi-agent pipeline with critic-guided iteration incurs high computational overhead, a known trade-off of iterative retrieval approaches (14), with mean latency reaching 9-12 seconds (and up to 93 seconds with reranking), making it significantly slower than single-pass dense RAG.

7 Conclusion

We presented **Agent-Enhanced GraphRAG**, a fully modular multi-agent pipeline designed to address the core limitations of standard RAG for multi-hop question answering. By decomposing complex questions into hop-by-hop retrieval plans, grounding evidence collection in dynamically constructed entity-tracked knowledge graphs, iteratively refining retrieval under critic guidance, and optionally learning precision-oriented passage reranking from entity-overlap supervision, the system achieves consistent improvements over both single-pass dense RAG (B1) and unorchestrated basic GraphRAG (B2) across three established multi-hop benchmarks.

Across the Qwen2.5-7B backbone (Run 1), OURS surpasses B1 by 9.1% higher EM on HotpotQA (36% vs. 33%) and 3.6% higher EM on 2WikiMultiHopQA (28.5% vs. 27.5%), and exceeds B2 by margins as large as 75.6% higher EM on HotpotQA and 171.4% higher EM on 2WikiMultiHopQA. With the trained passage reranker under disjoint splits (Run 3), the most compelling result emerges on 2WikiMultiHopQA, where OURS achieves 14.3% higher EM than B1 (36.0% vs. 31.5%) and 5.1% higher F1 (43.84% vs. 41.69%) - the strongest relative advantage observed in any single dataset-run combination in the study.

The three-run experimental design reveals three consistent patterns. First, the gains of multi-agent orchestration are backbone-partially-independent: even with Llama3.2-3B (Run 2), OURS achieves 52.2% higher EM than B1 on HotpotQA (17.5% vs. 11.5%) and 1,100% higher EM on MuSiQue (6.0% vs. 0.5%), confirming that the agentic pipeline extracts measurable value from a weaker generator. Second, larger and more capable LLM backends amplify those gains substantially, as evidenced by the 51.4% higher HotpotQA EM when moving from the 3B to the 7B model. Third, entity-aware passage reranking under strict generalization conditions provides targeted precision improvements on datasets with explicit relational structure like 2WikiMultiHopQA, without degrading performance on other datasets.

The primary trade-off is latency: OURS incurs 4-5× higher inference time than B1 in standard runs and up to 25-45× in the reranker-augmented run. Future work should pursue parallelized retrieval stages, cached KG construction, and distilled critic agents to bring latency within interactive bounds. Beyond efficiency, promising directions include scaling evaluation to full benchmark splits, replacing the closed relation vocabulary with open information extraction, incorporating structured output constraints for robustness with smaller LLMs, and extending the pipeline to multilingual and cross-lingual multi-hop settings. The results of this study support the conclusion that agent-level decomposition (9; 11) combined with critic-guided iterative retrieval (3) is a structurally sound and interpretable approach to multi-hop reasoning - one whose full potential will become more apparent as accessible open-source LLMs continue to scale.

Future directions include:

- Scaling to full benchmark evaluation with distributed inference,
- Replacing the closed relation vocabulary with open-information-extraction relations,
- Incorporating structured output constraints for LLM agents to eliminate JSON failure modes,
- Extending the pipeline to multilingual and cross-lingual multi-hop settings.

Acknowledgements

The authors thank the open-source communities behind HuggingFace Transformers, sentence-transformers, LangChain, NetworkX, and FAISS, as well as the Ollama project for enabling accessible local LLM inference.

References

[1] Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., and Larson, J. (2024). From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*.

[2] Feng, Y., Chen, X., Lin, B. Y., Wang, P., Yan, J., and Ren, X. (2020). Scalable multi-hop relational reasoning for knowledge-aware question answering. In *Proc. EMNLP*, pages 1295-1307.

[3] Gou, Z., Shao, Z., Gong, Y., Shen, Y., Yang, Y., Huang, M., and Duan, N. (2023). CRITIC: Large language models can self-correct with tool-interactive critiquing. *arXiv preprint arXiv:2305.11738*.

[4] Ho, X., Duong-Nguyen, A.-K., Sugawara, S., and Aizawa, A. (2020). Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In *Proc. COLING*, pages 6609-6625.

[5] Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y., Madotto, A., and Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1-38.

[6] Jiang, Z., Xu, F. F., Gao, L., Sun, Z., Liu, Q., Dwivedi-Yu, J., Yang, Y., Callan, J., and Neubig, G. (2023). Active retrieval augmented generation. In *Proc. EMNLP*, pages 7969-7992.

[7] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-T., Rocktäschel, T., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proc. NeurIPS*, pages 9459-9474.

[8] Mallen, A., Asai, A., Zhong, V., Das, R., Hajishirzi, H., and Weld, D. (2023). When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In *Proc. ACL*, pages 9802-9822.

[9] Min, S., Zhong, V., Zettlemoyer, L., and Hajishirzi, H. (2019). Multi-hop reading comprehension through question decomposition and rescoring. In *Proc. ACL*, pages 6097-6109.

[10] Peng, B., Zhu, Y., Liu, Y., Bo, X., Shi, H., Hong, C., and Tang, J. (2024). Graph-RAG: Leveraging knowledge graphs for more reliable and accurate information retrieval. *arXiv preprint arXiv:2406.15998*.

[11] Press, O., Zhang, M., Min, S., Schmidt, L., Smith, N. A., and Lewis, M. (2022). Measuring and narrowing the compositionality gap in language models. *arXiv preprint arXiv:2210.03350*.

[12] Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proc. EMNLP-IJCNLP*, pages 3982-3992.

[13] Saxena, A., Kochsiek, A., and Gemulla, R. (2022). Sequence-to-sequence knowledge graph completion and question answering. In *Proc. ACL*, pages 2814-2828.

[14] Shao, Z., Gong, Y., Shen, Y., Huang, M., Duan, N., and Chen, W. (2023). Enhancing retrieval-augmented large language models with iterative retrieval-generation synergy. In *Proc. EMNLP Findings*, pages 9248-9274.

[15] Sun, H., Dhingra, B., Zaheer, M., Mazaitis, K., Salakhutdinov, R., and Cohen, W. (2018). Open domain question answering using early fusion of knowledge bases and text. In *Proc. EMNLP*, pages 4231-4242.

[16] Trivedi, H., Balasubramanian, N., Khot, T., and Sabharwal, A. (2022a). MuSiQue: Multi-hop questions via single-hop question composition. *Transactions of the ACL*, 10:539-554.

[17] Trivedi, H., Balasubramanian, N., Khot, T., and Sabharwal, A. (2022b). Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proc. ACL*, pages 10014-10037.

[18] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. In *Proc. NeurIPS*, pages 24824-24837.

[19] Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W., Salakhutdinov, R., and Manning, C. D. (2018). HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proc. EMNLP*, pages 2369-2380.

[20] Yasunaga, M., Ren, H., Bosselut, A., Liang, P., and Leskovec, J. (2021). QA-GNN: Reasoning with language models and knowledge graphs for question answering. In *Proc. NAACL-HLT*, pages 535-546.

[21] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2022). ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.

[22] Johnson, J., Douze, M., and Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535-547.

[23] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

[24] Hagberg, A. A., Schult, D. A., and Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, pages 11-15.